Diretrizes Fase 1 — Suporte a Vídeo no PictuRAS

Relatório da Fase 1 e respetivo *template*

Versão 1

Requisitos e Arquiteturas de Software

Mestrado em Engenharia Informática

Universidade do Minho

17 de outubro de 2026

Objetivo

Este documento orienta a elaboração do **relatório da Fase 1** do projeto PictuRAS, cujo objetivo é estender a aplicação com funcionalidades de manipulação de vídeo digital, numa lógica de **MVP**. A título de sugestão, é também disponibilizado um *template* Typst (`src-doc-ext-requisitos-template.zip`) que segue a estrutura aqui recomendada, com tabelas pré-formatadas e caixas de orientação. A sua utilização é **opcional**: serve apenas para facilitar o trabalho.

Tal como definido no enunciado, a Fase 1 inclui, para além do levantamento de requisitos, **alterações arquiteturais**, **produção de código assistida por agentes de IA**, **integração** e **testes**. O relatório deve, por isso, fechar o ciclo *caso de uso* → *requisito* → *componente* → *teste*.

> **Entrega da Fase 1: 23 de outubro de 2026** (adiada de 16 de outubro).
> **Defesas: 26–30 de outubro de 2026**.

Princípios

- **Extensão, não reescrita.** O ponto de partida é a solução disponibilizada: documento de requisitos (`doc-ext-requisitos.pdf`), documento de solução arquitetural (`doc-sol-arquitetural.pdf`) e repositório base. O relatório descreve apenas o que muda; os elementos da *baseline* são referidos pelos seus IDs (UCnn, RFnn, RNFnn) quando são alterados.
- **MVP.** O foco é a validação funcional do suporte a vídeo. Isso não dispensa especificar requisitos não funcionais — dispensa apenas atingir já o nível de produção.
- **Qualidade em vez de quantidade.** Não é fixado um número de funcionalidades. Valoriza-se a profundidade, a complexidade adequada e a coerência de cada funcionalidade, bem como a sua implementação e verificação efetivas.
- **Rastreabilidade ponta a ponta.** Cada requisito tem origem (elemento do caso de uso, regra ou decisão de qualidade), é realizado por componentes identificados e verificado por testes.
- **IA documentada e validada.** Ferramentas, *skills*, tarefas apoiadas e validação feita pela equipa são documentadas, tanto na especificação como no código. A equipa é responsável por todo o conteúdo entregue.

Escolha das funcionalidades de vídeo

- O número de funcionalidades fica ao critério da equipa. Cada funcionalidade deve ter complexidade adequada. Operações triviais (e.g., gerar uma miniatura a partir de um único fotograma) não são suficientes.

- Requisitos comuns a várias funcionalidades (e.g., submeter vídeo, acompanhar o estado de um trabalho de processamento) são especificados **uma só vez**, indicando como origem os vários casos de uso — não se duplicam.
- Funcionalidades especificadas mas não implementadas devem estar claramente identificadas como tal (Cap. 3.1 e Cap. 7.3).

Estrutura do relatório

Recomenda-se a organização do relatório em três partes. Parte I: visão transversal. Parte II: especificação das funcionalidades de vídeo, organizada por artefacto — cada capítulo reúne todas as funcionalidades. Parte III: arquitetura, IA e validação. Trata-se de uma recomendação (é a estrutura seguida pelo *template*): a organização pode ser adaptada.

Elementos iniciais

- Capa (grupo e elementos), índices e, opcionalmente, histórico de versões.

Parte I — Enquadramento e visão transversal

1. Introdução. Propósito do incremento; *baseline* usada (documentos e *commit* de partida do repositório) e versão entregue; resumo do que foi feito; organização do documento.

2. Contexto, perfis e *stakeholders* (revisão). Apenas o que muda com o vídeo: limites por perfil (duração, tamanho, resolução, contagem de operações), novos *stakeholders* ou novos interesses. Os nomes dos *stakeholders* devem coincidir com os usados nos casos de uso. Um parágrafo sobre a elicitação: como chegaram às funcionalidades, regras e valores-alvo, e com quem; os valores numéricos dos requisitos devem ter origem identificável.

3. Visão geral da extensão de vídeo.

- 3.1 Tabela-resumo das funcionalidades (caso de uso, prefixo(s) dos requisitos, perfis, dependências, estado de implementação) e justificação de cada uma no contexto do MVP.
- 3.2 **Diagrama de casos de uso atualizado** — ponto crítico (ver secção seguinte).
- 3.3 **Modelo de domínio atualizado** — ponto crítico (ver secção seguinte).
- 3.4 Regras de negócio transversais (RNG-nn).
- 3.5 Convenções de identificação.

Parte II — Especificação das funcionalidades de vídeo

Cada capítulo reúne todas as funcionalidades; nas tabelas, cada linha indica o caso de uso a que se refere.

4. Casos de uso. Um caso de uso (UC-VID-nnn) por funcionalidade: cabeçalho, fluxo principal (ator/sistema), fluxos alternativos, exceções, pós-condições (sucesso e mínima), regras de negócio, assunções e questões em aberto com resolução.

5. Especificação de requisitos do sistema (ERS). 5.1 requisitos funcionais (prioridade, NOVO/ALT, origem, justificação, critério de aceitação *Dado* ... *Quando* ... *Então* ...); 5.2 requisitos não funcionais classificados segundo a ISO/IEC 25010; 5.3 verificação da completude e dos limites do âmbito de cada caso de uso.

6. Impacto no sistema existente. Elementos do PictuRAS existente e do documento de requisitos de base afetados (Manter / Estender / Alterar), com os casos de uso e requisitos (ALT) relacionados; eventuais conflitos entre funcionalidades resolvidos.

7. Rastreabilidade. 7.1 matriz elemento do caso de uso → requisitos, sem órfãos nem elementos por cobrir; 7.2 decisões de qualidade (DQ-nnn); 7.3 realização e verificação: para cada requisito, componente(s), estado (implementado / parcial / não implementado), teste(s) e resultado.

Parte III — Arquitetura, IA e validação

8. Alterações arquiteturais. Apenas o *delta* face à solução de base: vista de componentes atualizada, opcional (novos e alterados destacados); tabela de componentes com responsabilidades e requisitos; decisões arquiteturais (DA-nn) com contexto, alternativas e consequências; diagrama de sequência do fluxo de processamento de um vídeo; impacto na instalação e execução.

9. Implementação assistida por agentes de IA. Ferramentas, modelos e configuração (*skills*, instruções de projeto, ...); tarefas apoiadas (especificação, código, testes), validação feita pela equipa e evidência (*commits*, *pull requests*); análise crítica.

10. Testes e validação do MVP. Estratégia e ambiente; testes realizados (ID, tipo, descrição *Dado* ... *Quando* ... *Então* ..., requisitos, resultado); limitações e defeitos conhecidos.

11. Manutenção corretiva da solução de base (recomendado). Problemas encontrados na *baseline* durante o estudo e a integração (T-nn), com impacto, prioridade, requisitos relacionados e estado.

12. Conclusão. Contributos, o que ficou por fazer e porquê, lições aprendidas e ligação à Fase 2 (que requisitos não funcionais consideram prioritários).

Anexos. Instruções de execução da solução e dos testes; glossário.

Pontos críticos a atualizar

Alguns artefactos da *baseline* são centrados em imagem e têm de ser revistos. O *template* inclui as versões da solução de base, marcadas "a substituir", e caixas tracejadas para as novas versões.

Modelo de domínio. O modelo de base gira em torno de `Imagem`. Questões a que o modelo atualizado deve responder:

- Como se relaciona `Vídeo` com `Imagem` — especialização de um conceito comum ou conceito independente? Que atributos de domínio interessam (duração, resolução, formato, ...)?
- Um projeto passa a conter vídeos? Pode misturar imagens e vídeos?
- O processamento é demorado: existe um conceito de *trabalho de processamento* com estado e resultado?
- Que ferramentas se aplicam a vídeo? As de imagem aplicam-se a fotogramas? Como se representam quotas e limites por perfil?

O modelo de domínio representa conceitos do problema — não classes, tabelas nem microserviços — e usa a mesma terminologia dos casos de uso e requisitos.

Diagrama de casos de uso. Deve incluir todos os UC-VID-nnn, distinguindo-os visualmente; associar cada caso de uso aos perfis que lhe acedem, de forma coerente com as regras de negócio.

Perfis, quotas e *stakeholders*. A regra "até 5 operações por dia" talvez deva ser reinterpretada para vídeo (uma operação por vídeo? por minuto? por fotograma?). A decisão é transversal (RNG no Cap. 3.4) e todas as funcionalidades a devem respeitar.

Requisitos da *baseline* alterados. Sempre que um requisito novo altera um comportamento existente (ALT), o elemento da *baseline* afetado (UCnn, RFnn, RNFnn) é identificado no capítulo de impacto (Cap. 6).

Convenções de identificação

Identificador	Utilização
`UC-VID-nnn`	Caso de uso de vídeo; numeração única no relatório, a partir de UC-VID-001.
`REQ-VID-{FUNC}-nnn`	Requisito; {FUNC} identifica a funcionalidade ou a área do serviço (e.g., `REQ-VID-TRIM-001`, `REQ-VID-SUBMIT-001`, `REQ-VID-STATUS-001`). Um ID nunca se reutiliza, mesmo que o requisito seja eliminado.
`FA`k, `RN`k, `A`k, `Q`k	Fluxo alternativo, regra, assunção e questão, locais ao caso de uso; nas tabelas partilhadas (ERS, impacto, rastreabilidade) escrevem-se qualificados (`UC-VID-001/RN2`).
`RNG-nn`, `DQ-nnn`	Regra de negócio transversal; decisão de qualidade (numeração global).
`DA-nn`, `TST-{FUNC}-nn`, `T-nn`	Decisão arquitetural; teste; tarefa de manutenção corretiva.
`UCnn`, `RFnn`, `RNFnn`	Elementos do documento de requisitos de base, referidos quando alterados.

Template Typst (opcional)

O *template* é uma sugestão para quem o considere útil; não é obrigatório usá-lo, nem usar Typst.

- Compilação: `typst compile --font-path fonts main.typ doc-ext-requisitos.pdf` (ou carregar o conteúdo do zip em `typst.app`).
- Organização dos ficheiros:
  - `main.typ` — capa (grupo e elementos) e ordem dos capítulos;
  - `capitulos/` — um ficheiro por capítulo;
  - `casos-uso/UC-VID-001.typ` — um caso de uso; para cada novo caso de uso, copiar este ficheiro (e.g., `UC-VID-002.typ`) e acrescentar o `#include` no capítulo 4;
  - as tabelas 5.1 e 5.2 estão em páginas na horizontal (`#paisagem`).
- Texto a substituir aparece a vermelho entre chavetas. As caixas laranja de **Orientação** explicam cada secção e desaparecem com `#mostrar-orientacoes.update(false)` em `main.typ`.
- Tudo pode ser adaptado (estrutura, secções, aspeto gráfico).

Entregáveis

Entregável	Descrição	Ficheiro
Novo documento de requisitos (extensão)	Relatório descrito neste documento	`doc-ext-requisitos.pdf`
Fonte do documento de requisitos	Documento de extensão de requisitos em formato editável (e.g., Typst)	`src-doc-ext-requisitos.zip`
Solução estendida (MVP)	Implementação do MVP de vídeo integrada no PictuRAS	`solucao-ext-mvp.zip`
Evidências de uso de IA	Configuração dos agentes (*skills*, instruções) e *prompts* relevantes	No doc. e na solução, e.g.: `.claude/skills/<skill-name>/SKILL.md`
Instruções de execução	Como lançar a solução e correr os testes	Anexo do relatório e/ou `README`
Constituição do grupo	Elementos do grupo e performance individual com a estrutura do `grupo.csv` disponibilizado	`grupo.csv`

Lista de verificação antes de entregar

- ☐ Todas as funcionalidades listadas no Cap. 3.1, com estado de implementação.
- ☐ Diagrama de casos de uso e modelo de domínio atualizados para vídeo (as figuras de *baseline* foram substituídas).
- ☐ Cada caso de uso tem fluxo principal e pós-condições de sucesso.
- ☐ Requisitos atómicos, verificáveis e no padrão "O sistema deve. . . "; RNF classificados (ISO/IEC 25010).
- ☐ Nenhum requisito órfão; nenhum elemento relevante do caso de uso por cobrir; requisitos partilhados não duplicados.
- ☐ Elementos da *baseline* alterados (ALT) identificados no Cap. 6, sem conflitos entre funcionalidades.
- ☐ Alterações arquiteturais e decisões (DA) ligadas a requisitos.
- ☐ Cada requisito implementado tem pelo menos um teste com resultado registado.
- ☐ Utilização de agentes de IA documentada, com validação e evidência.
- ☐ `grupo.csv` com uma linha por elemento do grupo; cada performance individual no intervalo $[-2, +2]$ e somatório igual a 0.
- ☐ Se usarem o *template*: caixas de orientação escondidas e marcadores `{...}` removidos.